

Presented to the College of Computer Studies

De La Salle University - Manila

Term 1, A.Y. 2026-2027

In partial fulfillment of the course

CEPARCO S02

Project Proposal

Group 4

Submitted by:

Esperanza, MJ

Go, John William D.

Obar, David

Placer, Paul John C.

Submitted to:

Mr. Roger Luis Uy

October 19, 2026

ABSTRACT

Convolution is a mathematical operation that combines an input signal with a kernel/filter to produce an output, with applications in signal processing and image processing. Direct convolution will be implemented to study the impact of parallelism in programming at the data level. C programming with vectorization using AVX2 will be compared to a baseline implementation in scalar C for efficiency. Unlike others in the field, this project will focus on the effects of variation in input and kernel sizes on the speed and efficiency of programming. Different configurations will be tested. Execution times and calculated speedup values for both programming techniques will be presented and analyzed along the computed convolution arrays. A performance evaluation and visual aid will be presented. The visual aid will include a heat map in two dimensions showing performance improvement of SIMD vectorization across different sizes.

Keywords: Convolution, SIMD, AVX2, Performance Analysis

I. Proposed Project

Convolution will be implemented using AVX2 SIMD assembly to calculate multiple output elements simultaneously. The following inputs for this project include an input signal (x) and a convolution kernel or filter (h). The input signal must contain at least greater or equal elements as the kernel where $N \geq M$. The implementation will use valid convolution, producing an output of length (N-M+1), where (N) is the input length and (k) is the kernel length.

$$y[n] = \sum_{k=0}^{M-1} x[k]h[n-k]$$

Where:

$y[n] = output$

$x[n] = input$

$h[n] = filter/kernel$

Although parallelization of convolution has already been implemented, multiple works have examined its performance using different approaches and implementations, such as SIMD vectorization, loop reordering, and register blocking (Kelefouras & Keramidas, 2022; Barrachina et al., 2023). Kelefouras and Keramidas (2022) evaluated their implementation using 3×3, 5×5, 7×7, and 9×9 kernels across different image sizes. While these studies examine optimization techniques for convolution, this project will focus on analyzing how different combinations of

input and kernel sizes affect the performance of AVX2 convolution compared with a scalar implementation.

The convolution will be parallelized using AVX2 SIMD assembly, allowing multiple output elements to be calculated simultaneously. The implementation will use 256bit YMM registers to process eight single precision floating-point values at a time. The program will calculate eight output elements at once by multiplying the input values by the corresponding kernel coefficients and summing the results. Any remaining output elements will be calculated separately.

Input generation, memory allocation, correctness checking, and execution time measurement will be performed sequentially. The scalar C implementation will serve as the baseline for comparison with the AVX2 implementation. The project will produce convolution output arrays and execution time measurements for both implementations. Their performance will be compared using speedup values, which will be visualized through two dimensional heatmaps across different input and kernel sizes.

II. Target Platform

The project will use C for the baseline convolution implementation, correctness checking, and performance measurement. AVX2 assembly will be used to implement SIMD based convolution, allowing multiple output elements to be processed simultaneously. The program will target x86-64 processors that support AVX2 instructions.

III. Mid project milestones

The goal of the mid-project milestones is to finish the implementation of the program in AVX2 assembly, and developing a scalar C implementation to verify correctness and compare execution times. This will allow data analysis to begin and the final report to be prepared.

IV. AI generation usage

AI was used for idea generation such as literature review search, and grammar checking.

V. References

Barrachina, S., Castelló, A., Dolz, M. F., Low, T. M., Martínez, H., Quintana-Ortí, E. S., Sridhar,

U., & Tomás, A. E. (2023). Reformulating the direct convolution for high-performance deep learning inference on ARM processors. Journal of Systems Architecture, 135, Article 102806. https://doi.org/10.1016/j.sysarc.2022.102806

Kelefouras, V., & Keramidas, G. (2022). Design and implementation of 2D convolution on x86/x64 processors. IEEE Transactions on Parallel and Distributed Systems, 33(12), 3800–3815. https://doi.org/10.1109/TPDS.2022.3171471